

Ansible – Introdução



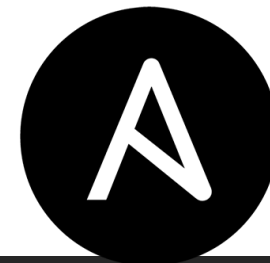
ANSIBLE



DevOps
Mão na
Massa

O que é e para que serve?

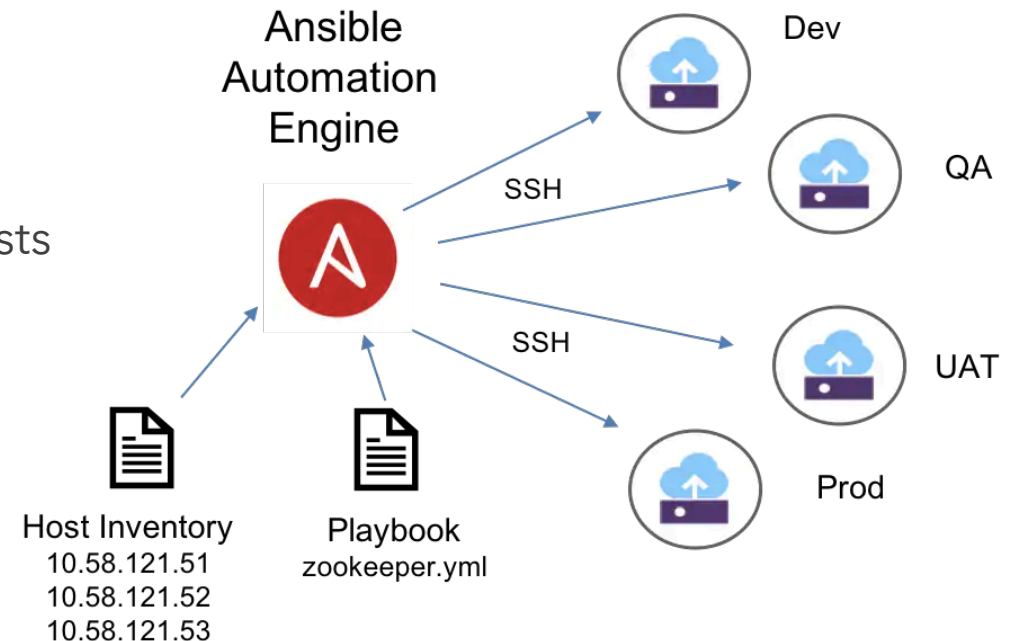
- Utilitário (open source) para orquestração e gerenciamento de configuração.
- Também conhecido como IaC (Infraestrutura como código)
- Automatização e padronização de configurações de hosts virtuais (bare metal e máquinas virtuais)
- Arquitetura simples e de fácil uso



ANSIBLE

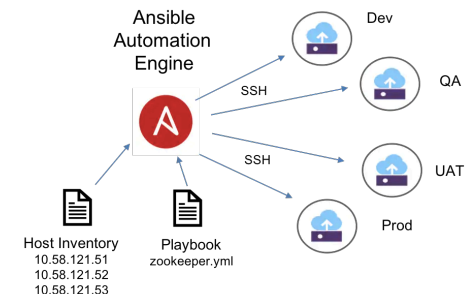
Ansible - Arquitetura

- Dois tipos de máquina: control node e managed hosts
- Control Machine: Instalação do Ansible
- Managed Host: Host inventory
- Host Inventory: arquivo texto (control node) com a lista de hosts / Ips que serão gerenciados pelo Ansible
- Forma de comunicação: SSH
- Nenhum software precisa ser instalado no Managed Host



Ansible – Arquitetura

- **Core modules:** executam grande maioria das atividades de adm. do sistema operacional.
- **Custom Modules:** estender a funcionalidade criando componentes personalizados (Python).
- **Playbooks:** Arquivo texto do tipo YAML com sintaxe pré-definida para configurar módulos Ansible.
- **Plugins:** Extensão de funcionalidades (envio de mensagens, email, etc).
- **Host Inventory:** arquivo texto (control node) com a lista de hosts / Ips que serão gerenciados pelo Ansible.
- **Ansible Galaxy:** Site que disponibiliza um conjunto de roles (tarefas) desenvolvidas pela comunidade.
- **Forma de comunicação:** SSH
- Nenhum software precisa ser instalado no Managed Host



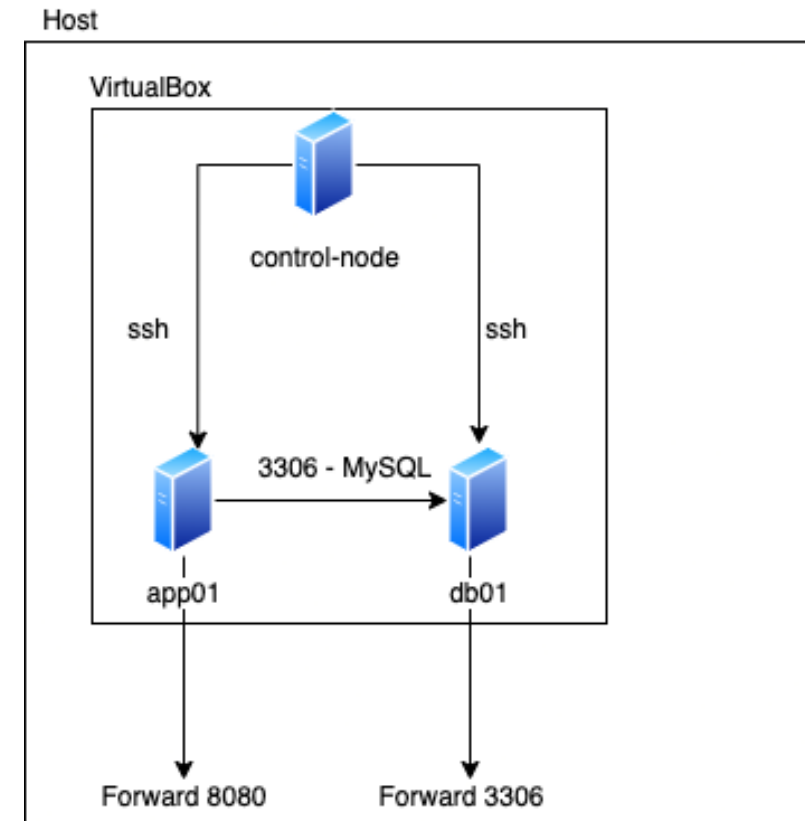
Ansible – Papéis – requerimentos S.O

- **Control Node:** DevOps (sysadmins) acessam (se for o caso) e inicializam operações através do Control Node.
- **Requerimentos Control Node:**
 - Python (2.7 ou 3.5 ou superior)
 - Redhat, Cent OS, Debian, macOS (BSD like) etc. Windows não é suportado.
- **Managed Host:** Log in, Instalação de módulos e execução dos comandos remotamente.
- **Requerimentos Managed Host:**
 - SSH habilitado
 - Python 2.4 – superior



Ansible - Lab - Arquitetura

- **Control Node:** Ansible e playbooks
- **APP01:**
 - Java - OpenJDK - Notes App (REST)
 - Maven
- **DB01:** MySQL



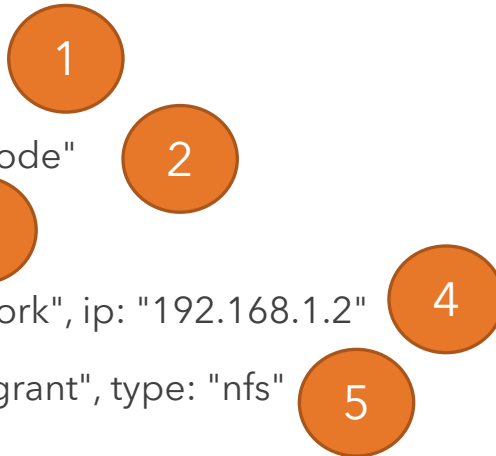
Ansible – Lab – Mão na Massa

- Criação de três servidores:
 - control-node
 - app01
 - db01
- Configuração de rede privada – todos os hosts
- Configuração do hostname – todos os hosts
- Configuração de forward port para aplicação 8080 – **app01**
- Configuração de forward port para db 3306 – **app01**
- Instalação do Ansible – **control-node**

Ansible – VagrantFile control-node

control-node

```
Vagrant.configure("2") do |config|  
  config.vm.hostname = "control-node"  
  config.vm.box = "centos/7"  
  config.vm.network "private_network", ip: "192.168.1.2"  
  config.vm.synced_folder ".", "/vagrant", type: "nfs"  
end
```



1. Inicio da configuração (V2)
2. Nome do hostname
3. Nome da box
4. Configuração rede interna
5. Configuração sync tipo nfs

Ansible – VagrantFile – APP01

app01

```
Vagrant.configure("2") do |config|  
  config.vm.hostname = "app01"  
  config.vm.box = "centos/7"  
  config.vm.network "private_network", ip: "192.168.1.3"  
  config.vm.network "forwarded_port", guest: 8080, host: 8080  
end
```

1

2

3

4

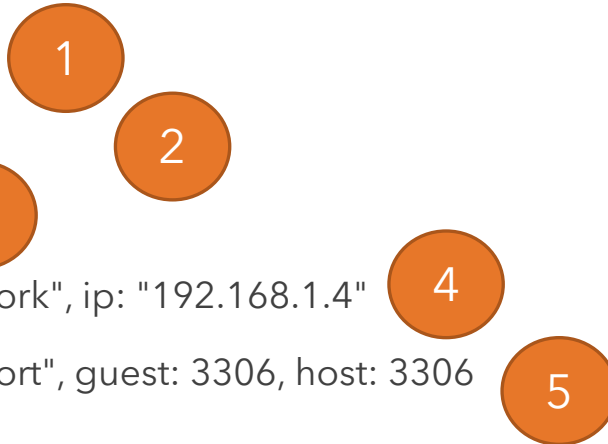
5

1. Inicio da configuração (V2)
2. Nome do hostname
3. Nome da box
4. Configuração rede interna
5. Forward da porta 8080

Ansible -VagrantFile - DB01

db01

```
Vagrant.configure("2") do |config|  
  config.vm.hostname = "db01"  
  config.vm.box = "centos/7"  
  config.vm.network "private_network", ip: "192.168.1.4"  
  config.vm.network "forwarded_port", guest: 3306, host: 3306  
end
```



1. Inicio da configuração (V2)
2. Nome do hostname
3. Nome da box
4. Configuração rede interna
5. Forward da porta 3306 - MySQL

Ansible - Lab - estrutura de diretórios

```
[macbook:lab-ansible glaucioguerre$ tree
```

```
.
├── app01
│   └── Vagrantfile
├── control-node
│   ├── Vagrantfile
│   └── playbooks
└── db01
    └── Vagrantfile
```

- Criar estrutura de diretórios
- Executar: **vagrant init** em cada diretório referente ao server.

Ansible – Configuração control node

- Instalação do Ansible
 - Configuração do repositório epel
 - **sudo yum install epel-release -y**
 - Instalação do ansible via yum
 - **sudo yum install ansible -y**
 - Configurar /etc/hosts com servidores criados
 - Adicionar hosts ao inventário
 - Configurar acesso ssh via key-based authentication (no password)
 - Alterar config **PasswordAuthentication** para **yes** no arquivo **/etc/ssh/sshd_config** (restart do serviço)
 - **ssh-keygen**
 - **ssh-copy-id**
 - Executar: **ansible -m ping all** para testar conectividade

Ansible – Playbook – alto nível

- Criação Playbook APP:
 - Update do S.O
 - Instalação pacotes de S.O
 - Instalação do Java
 - Instalação do Maven
 - Template – arquivos de configuração
- Criação Playbook DB:
 - Instalação MySQL
 - Update do S.O
 - Instalação de pacotes de SO
 - Instalação banco MYSQL Server
 - Criação banco



```

1 ---
2 - name: Configurar servidor de banco de dados
3   hosts: db01
4
5   user: vagrant
6   become: yes
7   vars_files:
8     - vars/main.yml
9   vars:
10     - dbname: "db01"
11     - dbusername: "root"
12     - dbpassword: "devopsmaonamassa"
13   tasks:
14     - name: Configuração hosts lab ansible
15       lineinfile: line="{{item}}" path=/etc/hosts
16       with_items:
17         - 192.168.1.2 control-node
18         - 192.168.1.3 app01
19         - 192.168.1.4 db01
20   roles:
21     - configuracao-default-so
22     - role: geerlingguy.mysql

```



Ansible – Playbook DB

- Definição de variáveis
- Configuração hosts – Lab Ansible
- Utilização Ansible Galaxy – Instalação MYSQL
- Utilização de role – config básica de S.O (pacotes / update, etc.)

Ansible – Playbook App

- Definição de variáveis
- Configuração hosts – Lab Ansible
- Utilização de role – config básica de S.O (compartilhada)
- Adicionar usuário APP
- Instalação do Maven / Java – JDK 8
- Criação de estrutura de diretórios para Aplicação
- Clone do repositório git
- Configurar arquivo de propriedades (utilizar templates)
- Geração do pacote
- Registrar versão do pacote (utilizar o register)
- Configurar serviço SystemD
- Utilização de handlers



Ansible – Validar o Lab

- Easy Note é uma aplicação REST
 - Inserir um registro através do control-node:
curl -H "Content-Type: application/json" --data @note.json <http://app01:8080/api/notes>
 - Recuperar todos as notas:
 - **curl <http://app01:8080/api/notes>**
 - Apagar um registro:
 - **curl -X DELETE -H "Content-Type: application/json" http://app01:8080/api/notes/1**

